Secure Hybrid SaaS Integration for OPERA PMS

Solution Architecture Options — Revised

Table of Contents

1. Acronym Legend
2. Updated Constraints Summary
3. OPERA 5 On-Premises Capability Reference
4. Hybrid Read/Write Solutions (4 Options)
5. Comparison Matrix
6. Recommended Architecture

1. Acronym Legend

Acronym	Full Name	Description
CDC	Change Data Capture	A pattern that tracks and streams committed database changes at the engine level, typically via transaction or redo logs — zero polling
CQRS	Command Query Responsibility Segregation	An architectural pattern separating read and write operations into distinct models and pipelines
Kafka	Apache Kafka	A distributed event streaming platform used as an event bus; consumers can replay events from any offset in the retained log
LogMiner	Oracle LogMiner	An Oracle built-in utility for reading the redo and archive log; used by Debezium as the Oracle CDC source
MSK	Amazon Managed Streaming for Apache Kafka	AWS's fully managed Kafka service; a cloud-hosted alternative to self-managed Kafka clusters
OWS	OPERA Web Services	Oracle's SOAP/XML-based API for on-premises OPERA 5.x; the on-prem write channel equivalent of OHIP
OHIP	Oracle Hospitality Integration Platform	Oracle's certified REST API integration layer for OPERA Cloud; Oracle's recommended write channel for cloud OPERA deployments
OXI	OPERA Exchange Interface	OPERA's internal business event framework that publishes state changes (bookings, check-ins, modifications) to external subscribers
PII	Personally Identifiable Information	Any data that can identify an individual (name, passport, email, phone, date of birth) — subject to masking requirements under SC-2
SSE	Server-Sent Events	A unidirectional HTTP-based push mechanism from server to client; lighter than WebSocket for read-only streams

2. OPERA 5 On-Premises — Capability Reference

https://ubs-doc.vercel.app/docs/projects/badar-hms/Opera_Config

3. Hybrid Read/Write Solutions

Solution 1: OHIP REST API (Bidirectional) + MERN Backend + Cloud SQL DB

> **Applicability:** OPERA Cloud deployments only. On-premises OPERA 5.x cannot use OHIP — see Solution 3 or 4.

Architecture Summary: The MERN cloud service communicates with OPERA exclusively through **Oracle's OHIP REST API** — Oracle's certified integration layer for OPERA Cloud. The Node.js backend calls OHIP endpoints to read availability and reservation data and to write new bookings. **OHIP webhooks** push every OPERA state change to the MERN backend in real time, satisfying TR-2 with no polling. On receipt of each webhook event, the Node.js backend writes the change to the cloud SQL DB and pushes the update to connected React clients via SSE. The hotel's static IP is used to whitelist OHIP webhook delivery to a known source.

```
[OPERA Oracle DB]
        │
        ▼
[OHIP REST API — Oracle Managed]
        │ Webhook push (real-time, server-initiated)
        ▼
[Node.js Express Backend]
        │ OHIP OAuth 2.0
        ├── GET /reservations, /availability ──────────────► [OHIP]
        └── POST /reservations ──────────────────────────► [OHIP] ──► [OPERA Oracle DB]
        │
        ├──► [Cloud SQL DB — MySQL / PostgreSQL]
        │          (authoritative cloud store)
        │
        └──► SSE / WebSocket push ──► [React Frontend / Stakeholder Dashboard]
```

Sync Mechanism: OHIP webhooks are Oracle-managed server push — no polling, no agent. Every committed OPERA change (booking created, modified, cancelled; check-in; rate change) triggers an outbound HTTP POST from Oracle's infrastructure to the MERN backend's webhook endpoint. The backend processes the payload, writes to the cloud SQL DB, and emits an SSE event to all connected React clients.

Compliance Mapping:

- ☑ TR-1: Bidirectional — OHIP webhooks push reads; MERN calls OHIP for writes
- ☑ TR-2: OHIP webhooks are Oracle server push; zero polling anywhere in the pipeline
- ☑ TR-3: Static IP used to whitelist OHIP webhook origin; no raw DB port forwarding
- ☑ SC-1: OHIP enforces TLS 1.3; cloud SQL DB TLS 1.3 in transit and at rest
- ☑ SC-2: PII transmitted only where required for reservation operations
- ☑ SC-3: No direct Oracle DB credentials in MERN — all access mediated by OHIP
- ☑ SA-1/SA-2: MFA at MERN auth layer; stakeholder reads served from cloud SQL DB
- ☑ DB-1: Cloud SQL DB updated synchronously on every OHIP webhook event

Tech Stack: OHIP REST API, Node.js/Express, MySQL or PostgreSQL (RDS / Cloud SQL / Supabase), React, SSE **Complexity:** ★★★ Medium **Cost Profile:** Medium–High (OHIP subscription + cloud SQL DB instance)

Pros:

- No Oracle client library, no Oracle DB credentials, and no on-premises software is required in the MERN codebase — the integration surface is limited to OHIP's REST API
- OHIP webhooks are genuine server push; the entire pipeline is polling-free with no custom agent to maintain
- Oracle validates all writes through its own business logic layer — double-bookings, invalid rate codes, and conflicting reservation states are rejected before they touch the cloud SQL DB

- With Atlas removed, a single cloud SQL DB serves all read and write workloads; no dual-write complexity, no schema divergence between two cloud stores
- SSE from the Node.js backend is a zero-infrastructure real-time push mechanism — no message broker, no Change Streams subscription, no additional service
- Best long-term maintainability: Oracle absorbs OPERA schema changes into OHIP, insulating the MERN codebase from PMS version upgrades

On-Premises Changes Required:

- **Static IP:** The hotel must have or acquire a static IP from the ISP so that Oracle can whitelist the source of outbound OHIP webhook deliveries. No static IP means OHIP cannot reliably deliver webhooks. This requires a business broadband contract and typically incurs a monthly ISP surcharge.
- **No firewall changes:** OHIP webhooks are delivered outbound by Oracle to the MERN backend's HTTPS endpoint. The hotel firewall does not need any new inbound rules — Oracle's infrastructure initiates the connection to the cloud, not to the hotel.
- **No on-premises software:** No agent, no CDC process, no additional service needs to be installed at the hotel. This is the only solution with zero on-premises footprint beyond the existing OPERA installation.
- **OHIP licensing and Oracle account:** An active OHIP subscription and Oracle Hospitality support agreement must be in place. This is a commercial and procurement change, not a technical one, but it is a real prerequisite.

Cons:

- Strictly limited to OPERA Cloud; on-premises OPERA 5.x properties cannot use OHIP under any circumstances
- OHIP licensing is Oracle enterprise-priced and procurement can take weeks — the commercial lead time is a project risk
- OHIP's published webhook event types and write endpoints may not cover all reservation operations; complex group allotments, linked profiles, and package components may require workarounds or be unavailable
- OHIP webhook reliability is Oracle-managed infrastructure; an Oracle-side outage breaks real-time sync and the MERN backend has no fallback unless a catch-up polling mechanism is built as a failsafe — which conflicts with TR-2 in spirit if not in letter
- The MERN backend must implement OHIP OAuth 2.0 token refresh, webhook signature verification, and idempotent event processing — non-trivial integration boilerplate before any business logic is written

---

Solution 2: Direct Oracle DB + IPSec VPN + CDC Fan-Out to Cloud SQL DB

Architecture Summary: The MERN Node.js backend connects **directly to the OPERA Oracle database** over an **IPSec site-to-site VPN** terminated at the hotel's static IP. A dedicated Oracle DB user with scoped `SELECT`, `INSERT`, and `UPDATE` permissions on reservation tables handles all MERN-originated reads and writes. On the outbound sync path, **Oracle LogMiner or GoldenGate** captures every committed change at the redo log level — with zero polling — and streams it via a Debezium or GoldenGate agent to **Apache Kafka** in the cloud. A single Kafka consumer applies changes to the cloud SQL DB in sub-second time. The Node.js backend subscribes to the Kafka topic and pushes updates to React clients via SSE.

```
[MERN React Frontend] ◄── SSE push ──[Node.js Express Backend]
                                          │ node-oracledb TLS 1.3
                                          │ over IPSec VPN
                                          ▼
                              [Hotel Firewall — Static IP — VPN Endpoint]
                                          │
                                          ▼
                                    [OPERA Oracle DB]
                                SELECT / INSERT / UPDATE
                                          │
                                    [Redo log / archive log]
                                          │
                              [LogMiner / GoldenGate CDC Agent]
```

```
                                        │ TLS 1.3 outbound
                                        ▼
                              [Apache Kafka / AWS MSK]
                                        │
                                 [SQL DB Consumer]
                                        │
                         [Cloud SQL DB — MySQL / PostgreSQL]
```

Sync Mechanism: Debezium or GoldenGate reads the Oracle redo log continuously — no SQL queries, no polling, no OPERA session consumption. Every committed transaction is emitted as a Kafka event within milliseconds. The Kafka SQL consumer applies an `INSERT`/`UPDATE` to the cloud SQL DB. The Node.js backend subscribes to the Kafka topic (or listens to a database notification from the SQL DB) and emits SSE events to connected React clients.

Compliance Mapping:

- ☑ TR-1: Bidirectional — MERN writes directly to Oracle via VPN; CDC streams all changes out
- ☑ TR-2: CDC reads redo log continuously; zero polling in any component
- ☑ TR-3: Static IP required; used for VPN endpoint and Oracle listener whitelist
- ☑ SC-1: IPSec VPN + Oracle native TLS; Kafka TLS 1.3; cloud SQL DB TLS 1.3
- ☑ SC-2: PII masking applied in CDC transformation layer before Kafka events
- ☑ SC-3: Dedicated Oracle user scoped to `SELECT`/`INSERT`/`UPDATE`; no `DELETE` or DDL
- ☑ SA-1/SA-2: MFA at MERN auth layer; stakeholder reads from cloud SQL DB
- ☑ DB-1: Kafka SQL consumer applies changes to cloud SQL DB within milliseconds of every Oracle commit

Tech Stack: node-oracledb, Oracle LogMiner / GoldenGate, Apache Kafka / AWS MSK, Debezium, Node.js, MySQL or PostgreSQL (RDS), IPSec VPN, React, SSE **Complexity:** ★★★★ Medium–High **Cost Profile:** Medium–High (VPN hardware + GoldenGate license if used + Kafka cluster + SQL DB)

Pros:

- CDC at the redo log level is the most comprehensive sync mechanism available — it captures every committed Oracle transaction, including PMS back-office edits, walk-ins, and manual DB changes that no API would ever surface
- The MERN backend has full, unrestricted control over Oracle queries and write operations — no API layer limiting which tables or operations are accessible
- Kafka's durable log means the cloud SQL DB can be fully rebuilt by replaying from offset 0 after any outage, schema migration, or bug fix — without touching OPERA
- IPSec VPN is a well-understood, auditable network security control familiar to hotel IT teams and Oracle DBAs
- With Atlas removed, a single Kafka consumer → single SQL DB is a clean, debuggable pipeline with no dual-write complexity

On-Premises Changes Required:

- **Static IP:** Mandatory. The hotel must have a fixed public IP address for the VPN endpoint and Oracle TNS listener whitelist. Without it, the VPN tunnel cannot be reliably established and the MERN backend cannot maintain a stable connection to Oracle. Requires business broadband and an ISP surcharge.
- **IPSec VPN configuration:** The hotel router or firewall must be configured to terminate a site-to-site IPSec VPN tunnel with the cloud provider. This requires a business-grade router capable of IKEv2/IPSec (Cisco, Fortinet, Mikrotik, pfSense). Consumer-grade ISP routers cannot do this. Hotel IT or MSP must configure and maintain the tunnel, including renegotiation and monitoring.
- **Firewall inbound rule — VPN:** An inbound firewall rule must permit IKE (UDP 500) and IPSec NAT-T (UDP 4500) from the cloud provider's IP range to the hotel's VPN endpoint. This is the only required inbound rule — Oracle TNS (1521) must remain strictly internal and must not be routed through to the internet directly.
- **Oracle supplemental logging:** A hotel Oracle DBA must run `ALTER DATABASE ADD SUPPLEMENTAL LOG DATA` and enable table-level supplemental logging on the relevant OPERA tables. This is a one-time DBA task but requires Oracle DBA access and a brief scheduled maintenance window.

- **CDC agent installation (Debezium or GoldenGate):** A Debezium container or GoldenGate agent must be installed and maintained on a VM in the hotel LAN. This VM needs outbound access to the Kafka broker (TCP 9092 or 9093 with TLS) — an outbound firewall rule that most corporate firewalls already permit on port 443 if the Kafka broker uses MSK with TLS on port 9094.
- **Oracle DB user provisioning:** A DBA must create a dedicated integration Oracle user with `SELECT`/`INSERT`/`UPDATE` grants on the relevant OPERA tables, and a separate CDC log-mining user (`GRANT LOGMINING TO cdc_user`). Two named accounts, both auditable.
- **Dedicated integration VM:** A separate Windows Server or Linux VM is strongly recommended for both the CDC agent and connection proxy. This avoids resource contention on the OPERA application server.

Cons:

- Direct Oracle write credentials in the cloud MERN environment are the single highest-risk element in this document — a compromised Node.js backend has a live write path to the OPERA production database. Must be mitigated with a WAF, connection proxy, per-query audit logging, and a ≤90-day credential rotation policy
- GoldenGate requires an enterprise Oracle license — procurement and cost are significant. Debezium with LogMiner is free but demands Oracle DBA tuning expertise to avoid redo log bloat and CPU overhead on the OPERA server
- The on-premises footprint is the largest of any solution: static IP, VPN hardware, CDC agent VM, DBA provisioning, and supplemental logging all require coordinated hotel IT effort before a single line of MERN code can be written
- Any OPERA PMS upgrade that alters reservation table schemas can silently break MERN queries or CDC transformation mappings — a schema-change monitoring and release process must be in place
- Kafka adds a managed infrastructure layer with its own operational overhead (topic management, consumer group monitoring, retention policies, and scaling)

Solution 3: On-Prem Node.js Agent + WebSocket + Cloud SQL DB ★ Recommended

Architecture Summary: A **Node.js sync agent** runs on a dedicated VM within the hotel LAN. It connects to the OPERA Oracle DB locally using scoped credentials and maintains a **persistent WSS (WebSocket Secure) connection** outbound to the MERN cloud backend — requiring no inbound firewall rules of any kind. **Oracle DB triggers** fire synchronously on every committed `INSERT` or `UPDATE` to OPERA's reservation tables, invoking the agent immediately. The agent serialises the event payload, filters PII per SC-2, and pushes it up the WebSocket to the Node.js backend. The backend writes the change to the cloud SQL DB and emits an SSE event to connected React clients. For writes, the MERN backend sends reservation commands down the same WebSocket channel; the agent executes them locally against Oracle and acknowledges the result.

```
[OPERA Oracle DB]
        │ DB trigger fires on commit
        ▼
[On-Prem Node.js Agent] ◄── INSERT/UPDATE command (write path)
        │
        │ WSS (TLS 1.3) — outbound only, persistent
        │
        ▼
[MERN Node.js Backend]
        │
        ├──► [Cloud SQL DB — MySQL / PostgreSQL]
        │         (INSERT / UPDATE on event receipt)
        │
        └──► SSE push ──► [React Frontend / Stakeholder Dashboard]

[Write path detail]
[React Frontend] ──► [Node.js Backend] ──WSS command──► [Agent] ──► [OPERA Oracle DB]
                                                                  │
                                                        ack / confirmation back up WSS
```

Sync Mechanism: Oracle DB triggers are synchronous with the commit — the agent receives the event in the same transaction window, with no polling interval and no redo log configuration required. The WSS connection is full-duplex: OPERA changes flow upward to the backend, and write commands flow downward to the agent. No Kafka, no broker, no CDC configuration.

Compliance Mapping:

- ☑ TR-1: Bidirectional — agent pushes OPERA changes up WSS; backend sends write commands down WSS
- ☑ TR-2: DB trigger fires on commit, not on schedule; WSS delivers the event immediately — zero polling
- ☑ TR-3: Static IP not required; WSS is an outbound connection from the agent
- ☑ SC-1: WSS enforces TLS 1.3; Oracle local connection uses native Oracle encryption (`sqlnet.ora`)
- ☑ SC-2: PII filtered in agent before serialisation onto the WebSocket
- ☑ SC-3: Agent holds scoped Oracle credentials locally — `SELECT` for reads, `INSERT`/`UPDATE` on reservation tables for writes; credentials never leave the LAN
- ☑ SA-1/SA-2: MFA at MERN auth layer; cloud SQL DB serves all stakeholder reads
- ☑ DB-1: Cloud SQL DB updated synchronously on every WSS event receipt in the Node.js backend

Tech Stack: Node.js (on-prem agent), WebSocket (`ws` library), node-oracledb, Oracle DB triggers, Node.js (MERN backend), MySQL or PostgreSQL (RDS / Cloud SQL / Supabase), React, SSE **Complexity:** ★★★ Medium **Cost Profile:** Low–Medium (agent VM compute + cloud SQL DB instance; no additional Oracle licensing)

Pros:

- Oracle credentials never leave the hotel LAN — the cloud backend sends command payloads only; it has no DB connection string, no username, no password for Oracle
- The entire stack is Node.js, meaning the same team owns the on-prem agent and the MERN backend with no language or toolchain context switching
- DB triggers are synchronous with the Oracle commit — zero polling, zero latency gap between a reservation being saved in OPERA and the event being dispatched
- With Atlas removed, the cloud SQL DB is the single target; a simple `pg` or `mysql2` parameterised upsert in the webhook handler is the entire sync implementation
- SSE from Node.js is a native HTTP feature — no broker, no subscription service, no additional infrastructure for real-time React updates
- No static IP required — the WebSocket connection is outbound from the hotel; if the hotel's IP changes, the agent reconnects automatically
- Lowest total cost of any fully compliant solution: agent VM, cloud SQL DB, and standard Node.js libraries are the only requirements

On-Premises Changes Required:

- **Dedicated agent VM:** A dedicated Windows Server or Linux VM must be provisioned in the hotel LAN to run the Node.js agent. It must have network access to the OPERA Oracle DB (TCP 1521, internal only) and outbound internet access on TCP 443. Running the agent on the OPERA application server itself is strongly discouraged due to resource contention risk on the PMS.
- **Oracle DB triggers:** A hotel Oracle DBA must create `AFTER INSERT OR UPDATE` triggers on the relevant OPERA reservation tables (`RESERVATION`, `RESERVATION_NAME`, `ALLOTMENT`, and others as required). This is a one-time DBA task, requires no OPERA application restart, and adds negligible overhead per transaction. The triggers call a small PL/SQL procedure that notifies the Node.js agent via Oracle's `DBMS_AQ` or a lightweight notification channel.
- **Oracle DB user provisioning:** A DBA must create a dedicated integration Oracle user with `SELECT`/`INSERT`/`UPDATE` grants on the reservation tables. Credentials are stored only on the agent VM — not in the cloud.
- **Outbound firewall rule (TCP 443):** The hotel firewall must permit outbound TCP 443 from the agent VM to the MERN backend's hostname. This rule is almost universally open by default on any internet-connected network; in practice no firewall change is typically needed.
- **No inbound firewall changes:** Because the WebSocket is outbound-initiated from the agent, the hotel firewall requires zero new inbound rules. This is the lowest on-premises network change requirement of any solution in this

document.
- **No static IP:** Not required. The cloud backend accepts WebSocket connections from any source IP.
- **Agent process management:** The agent process must be managed with a process supervisor (PM2, Windows Service wrapper, or systemd on Linux) to ensure automatic restart on crash or server reboot. Remote management tooling (SSH, RDP, or a fleet management tool) is recommended for multi-property deployments.

Cons:

- Oracle DB triggers add a small per-transaction overhead on every `INSERT`/`UPDATE` to the watched tables. On high-volume OPERA installations (large conference hotels, chains with high occupancy) this should be load-tested before production deployment
- The persistent WebSocket must survive hotel network instability — power cuts, ISP outages, and router reboots. The agent must implement exponential backoff reconnection and an in-memory event buffer to hold events during brief disconnections without data loss
- The on-prem Node.js agent is a custom-deployed component. Across a multi-property chain it must be versioned, packaged, and deployed consistently — a remote management and CI/CD strategy for on-prem agents is required at scale
- Write commands flowing MERN → WSS → Agent → Oracle introduce a two-hop write latency of typically 20–80ms additional round-trip (dependent on geographic distance between hotel and cloud region). For interactive booking flows this is imperceptible; for bulk operations it should be considered
- If the WebSocket drops mid-write, the agent may execute the Oracle write but fail to return the acknowledgement — idempotency keys on all write commands are mandatory to prevent duplicate reservation creation on retry

---

Solution 4: CDC + Kafka Event Bus + Cloud SQL Consumer + OHIP/OWS Writes

Architecture Summary: The most enterprise-grade option and the highest sync fidelity available. **Oracle GoldenGate or Debezium** captures every committed change in the OPERA Oracle DB at the redo log level — with zero polling, zero Oracle session consumption, and zero application-layer dependency — and streams events to **Apache Kafka** in the cloud. A single Kafka consumer applies every OPERA change to the cloud SQL DB within milliseconds. The Node.js backend subscribes to the Kafka topic and pushes updates to React clients via SSE. Write operations from the MERN frontend travel through **OHIP REST API** (for OPERA Cloud) or **OWS** (for on-premises OPERA 5), providing an Oracle-validated write channel. Once committed to Oracle, writes are re-captured by CDC and flow back through Kafka to the cloud SQL DB automatically — creating a closed, self-consistent sync loop with no separate confirmation write.

```
[OPERA Oracle DB]
        │ Supplemental logging on
        ▼
[GoldenGate / Debezium CDC Agent]
        │ TLS 1.3 — outbound to Kafka
        ▼
[Apache Kafka / AWS MSK — Event Bus]
        │
        ▼
[SQL DB Kafka Consumer]
        │
        ▼
[Cloud SQL DB — MySQL / PostgreSQL]
        │
[Node.js Backend] ── Kafka subscription ──► SSE push ──► [React Frontend]
        │
        │ Write path
        ▼
[OHIP REST API (OPERA Cloud)] ──► [OPERA Oracle DB]
   or
[OWS SOAP (OPERA 5 on-prem)] ──► [OPERA Oracle DB]
```

```
          │
          └──► CDC re-captures commit ──► Kafka ──► SQL Consumer ──► Cloud SQL DB
```

Sync Mechanism: CDC reads Oracle's redo log asynchronously at the storage level — it does not execute SQL queries and consumes no Oracle session slots. Every committed transaction reaches Kafka within milliseconds. The single Kafka SQL consumer maintains an ordered, idempotent stream of `INSERT`/`UPDATE` operations against the cloud SQL DB. MERN-originated writes go through OHIP or OWS, are committed to Oracle, re-captured by CDC, and surface in the cloud SQL DB through the normal consumer path — no additional write logic required.

Compliance Mapping:

- ☑ TR-1: Bidirectional — CDC streams OPERA changes outbound; OHIP/OWS handles writes
- ☑ TR-2: CDC is event-driven at redo log level; zero polling in the entire pipeline
- ☑ TR-3: Static IP recommended for CDC replication target and OWS port whitelist
- ☑ SC-1: GoldenGate TLS 1.3; Kafka TLS 1.3; OWS/OHIP TLS 1.3; cloud SQL DB TLS 1.3 — full chain
- ☑ SC-2: PII masking applied in the CDC transformation layer before Kafka events leave the hotel network
- ☑ SC-3: CDC log-mining user is read-only at the DB level; OHIP/OWS writes without DDL access
- ☑ SA-1/SA-2: MFA at MERN auth layer; cloud SQL DB serves all stakeholder reads
- ☑ DB-1: Kafka SQL consumer keeps cloud SQL DB within milliseconds of every OPERA commit

Tech Stack: Oracle GoldenGate or Debezium, Apache Kafka / AWS MSK, OHIP (OPERA Cloud) or OWS (OPERA 5), Node.js, MySQL or PostgreSQL (RDS), React, SSE **Complexity:** ★★★★★ High **Cost Profile:** High (GoldenGate license + Kafka cluster + OHIP subscription if applicable + SQL DB)

Pros:

- The only solution achieving sub-100ms sync to the cloud SQL DB for every OPERA transaction — including back-office edits, walk-ins, and manual DBA changes that no application API would ever surface
- Kafka's durable log means the cloud SQL DB can be fully rebuilt from scratch by replaying from offset 0 after any outage, schema migration, or consumer bug — without touching OPERA or requiring any on-premises action
- The write path (OHIP/OWS) and read path (CDC) are completely decoupled — an OHIP outage does not interrupt CDC sync, and a CDC agent restart does not affect write capability
- Additional consumers can be attached to the Kafka topic at any time (data warehouse, analytics pipeline, third-party tool) with zero changes to OPERA, the MERN backend, or the existing SQL consumer
- CDC log-mining user is read-only at the DB level — the highest SC-3 compliance posture of any solution, as the cloud service has no write credentials for Oracle at all on the CDC path
- With Atlas removed, a single Kafka consumer → single SQL DB is a clean, well-bounded pipeline

On-Premises Changes Required:

- **Static IP:** Strongly recommended. The CDC agent must stream outbound to a Kafka broker. While this connection is outbound (not requiring a static IP for the agent itself), the OWS write channel (for OPERA 5) requires the cloud to call inbound to the hotel's OWS port, which must be IP-whitelisted. Without a static IP, that whitelist breaks whenever the ISP rotates the hotel's address. Requires business broadband and ISP surcharge.
- **Oracle supplemental logging:** A DBA must run `ALTER DATABASE ADD SUPPLEMENTAL LOG DATA` and enable supplemental logging on all integration-relevant tables. This is a one-time change requiring a scheduled maintenance window. It increases redo log volume by 5–15% depending on transaction rate — archive log storage capacity must be assessed.
- **CDC agent installation (Debezium or GoldenGate):** A Debezium container (JVM-based) or GoldenGate agent must be installed on a dedicated VM in the hotel LAN. The VM needs outbound TCP 9092–9094 (Kafka/MSK with TLS) or TCP 443 if MSK is configured with a private endpoint — most firewalls permit outbound 443 by default.
- **GoldenGate licensing (if chosen):** GoldenGate requires purchasing an Oracle GoldenGate license and installing the GoldenGate software on-premises. This is a significant commercial and procurement effort. Debezium is the open-source alternative but requires more Oracle DBA tuning expertise.

- **Oracle CDC DB user:** A DBA must create a dedicated log-mining user: `CREATE USER cdc_user ...`, `GRANT CREATE SESSION TO cdc_user`, `GRANT LOGMINING TO cdc_user`, plus `SELECT` grants on `V$LOG`, `V$LOGFILE`, and relevant OPERA tables. This is separate from the integration read/write user.
- **Firewall inbound rule — OWS port (OPERA 5 only):** If using OWS as the write channel, the hotel firewall must permit inbound TCP on the OWS HTTP port (8080 or 443) from the MERN backend's cloud egress IP range. This must be source-IP-whitelisted — never open to `0.0.0.0/0`. For OHIP (OPERA Cloud), no hotel inbound rule is needed as OHIP is Oracle-managed.
- **Outbound firewall rule — Kafka:** Outbound TCP 9094 (MSK TLS) or 443 (MSK private endpoint) from the CDC agent VM to the Kafka broker must be permitted. This is an outbound rule and is generally open by default, but must be verified per property.
- **Dedicated CDC VM:** The Debezium or GoldenGate agent requires a dedicated VM — it must not run on the OPERA application server. Recommended: 4 vCPU, 8GB RAM minimum for Debezium with LogMiner on a moderately busy Oracle DB.
- **Archive log retention:** LogMiner requires Oracle archive logs to be retained for at least the duration of any expected CDC agent downtime. Standard OPERA deployments may have short retention windows — the DBA must extend archive log retention to at least 24–48 hours and ensure adequate disk space.

Cons:

- The largest on-premises change requirement of any solution: supplemental logging, archive log retention, CDC agent VM, dedicated Oracle DB user, GoldenGate licensing (or Debezium tuning), and — for OPERA 5 — an inbound firewall rule for OWS. This cannot be deployed without significant hotel IT and Oracle DBA involvement
- GoldenGate is enterprise-priced; procurement and installation can take as long as the OHIP licensing in Solution 1. Debezium with LogMiner is free but LogMiner is known to impact CPU and I/O on busy Oracle databases if `SUPPLEMENTAL_LOG_DATA` and archive log settings are not tuned correctly
- Kafka is a non-trivial managed infrastructure layer: topic configuration, consumer group management, retention policy, scaling, and monitoring are all ongoing operational concerns that sit outside the MERN team's typical stack
- Any OPERA upgrade changing reservation table structures requires updating the CDC transformation mapping before deployment — an Oracle-MERN release coordination dependency
- The full pipeline (Oracle → CDC → Kafka → SQL consumer → cloud SQL DB → SSE → React) is the deepest debugging surface in this document; correlating a missing or malformed event requires tracing across five system boundaries

---

4. Comparison Matrix

Solution	Write Channel	Sync Mechanism	Cloud DB	Polling?	Static IP	On-Prem Footprint	Complexity	Cost
1. OHIP Bidirectional	OHIP REST API	OHIP Webhooks	Cloud SQL DB	✗ None	Optional	None (zero on-prem install)	★★★	Med–High
2. Direct Oracle + CDC + Kafka	node-oracledb direct	CDC redo log → Kafka	Cloud SQL DB	✗ None	☑ Required	VPN + CDC VM + supplemental logging	★★★★	Med–High
3. WebSocket Agent ★	Agent → Oracle (local)	DB trigger → WSS	Cloud SQL DB	✗ None	Optional	Agent VM + DB triggers	★★★	Low–Med
4. CDC + Kafka + OHIP/OWS	OHIP or OWS	CDC redo log → Kafka	Cloud SQL DB	✗ None	Recommended	CDC VM + supplemental logging + archive log	★★★★★	High

Write Capability by Table

Solution	RESERVATION	RESERVATION_NAME	ALLOTMENT	RATE_HEADER	Direct DDL
1. OHIP	☑ OHIP-validated	☑ OHIP-validated	⚠ Limited by OHIP endpoints	✕	✕
2. Direct Oracle	☑ Full control	☑ Full control	☑ Full control	⚠ Explicit grant required	✕
3. WebSocket Agent ★	☑ Agent-mediated	☑ Agent-mediated	☑ Agent-mediated	⚠ Configurable	✕
4. CDC + OHIP/OWS	☑ OHIP or OWS	☑ OHIP or OWS	⚠ Depends on write channel	✕	✕

Cloud SQL DB Sync Latency

Solution	Latency	Mechanism
1. OHIP	< 1s	OHIP webhook → Node.js upsert
2. Direct + CDC	< 500ms	Redo log → Kafka consumer upsert
3. WebSocket Agent ★	< 200ms	DB trigger → WSS → Node.js upsert
4. CDC + Kafka	< 100ms	Redo log → Kafka consumer upsert

On-Premises Change Burden

Solution	Static IP	Inbound Firewall	New On-Prem Software	DBA Work	Effort Estimate
1. OHIP	Required (webhook whitelist)	None	None	None	Low — commercial procurement only
2. Direct + CDC	Required	VPN (UDP 500/4500 inbound)	CDC agent VM	High (supplemental logging, 2 DB users)	High — multi-week hotel IT + DBA engagement
3. WebSocket Agent ★	Not required	None	Agent VM	Low (DB triggers, 1 DB user)	Low–Medium — 1–2 days hotel IT + DBA
4. CDC + Kafka	Recommended	OWS port (OPERA 5 only)	CDC agent VM	High (supplemental logging, archive log, 2 DB users)	High — multi-week hotel IT + DBA engagement

5. Recommended Architecture

Primary Recommendation: Solution 3 (WebSocket Agent)

Solution 3 is the recommended starting point for any MERN-based team deploying against OPERA 5 on-premises. It satisfies every constraint with the lowest on-premises change burden, the lowest cost, and no Oracle licensing dependencies:

- Oracle credentials never leave the hotel LAN

- DB triggers provide zero-polling, zero-latency event dispatch on commit
- The cloud SQL DB is the single sync target — one `pg` or `mysql2` upsert in the backend handler
- SSE from Node.js drives real-time React updates with no additional infrastructure
- No static IP required; multi-property rollout is consistent and scriptable
- The entire stack — agent and backend — is Node.js, owned by one team

The on-premises installation reduces to: one VM, one set of DB triggers, one Oracle DB user, and one outbound firewall rule that is almost certainly already open.

Secondary Recommendation: Solution 1 (OHIP)

If the property is on **OPERA Cloud** and the OHIP subscription can be procured, Solution 1 is the architecturally cleanest option — zero on-premises footprint, Oracle absorbs all schema complexity, and the MERN team works purely against a REST API. The cloud SQL DB sync is a single webhook handler writing to one database.

For Enterprise Scale: Solution 4 (CDC + Kafka)

At 10+ properties, where millisecond sync fidelity, full event replay, and downstream extensibility to analytics or reporting pipelines are required, Solution 4 justifies its cost and on-premises complexity. The per-event latency and auditability are unmatched. This solution should not be chosen for a single-property deployment — the Kafka and CDC operational overhead is disproportionate below enterprise scale.

What to Avoid

Solution 2 (Direct Oracle + VPN) is a last resort. Direct Oracle write credentials in the cloud environment are an unacceptable risk to the OPERA production database, and it carries the second-highest on-premises change burden. If it must be used, it requires a WAF in front of the Node.js backend, a connection proxy between Node.js and Oracle (never a direct pool), per-query audit logging, and a maximum 90-day credential rotation policy enforced by policy, not convention.